

04b — JSON APIs

Key Concepts

Term	Definition
JSON	JavaScript Object Notation — lightweight text format for data exchange
JSON:API	Formal specification (jsonapi.org) for structuring JSON request/response bodies
Content-Type	HTTP header declaring the data format being sent
Serialization	Converting an object to JSON string for transmission
Deserialization	Parsing a JSON string back into an object

JSON vs XML

	JSON	XML
Readability	Human-friendly	Verbose, tag-heavy
Payload size	Smaller	Larger
Parsing	Native in JS, easy everywhere	Requires XML parser
Data types	String, number, boolean, array, object, null	Everything is text/tags
Comments	Not supported	Supported

```
// JSON
{ "name": "Anil", "roles": ["admin", "editor"] }
```

```
<!-- XML — same data -->
<user><name>Anil</name><roles><role>admin</role><role>editor</role></roles></user>
```

JSON Data Types

Type	Example
String	"name": "Anil"
Number	"age": 30
Boolean	"active": true
Array	"roles": ["admin", "editor"]

Type	Example
Object	"address": { "city": "Mumbai" }
Null	"deleted_at": null

Headers

```
Content-Type: application/json           // standard JSON
Content-Type: application/vnd.api+json  // JSON:API spec
Accept: application/json
```

Interview Questions

Q: Why did JSON replace XML for APIs? A: JSON is smaller, more readable, parses natively in JavaScript, and supports richer data types. XML is verbose and requires a dedicated parser.

Q: What data types does JSON support? A: String, number, boolean, array, object, and null.

Q: What is the limitation of JSON? A: No comment support, no schema enforcement (unless using JSON Schema), no distinction between integers and floats natively.